

# AI Dynamic Ride Pricing System

## 1.Problem Statement

Ride-sharing platforms dynamically adjust ride prices based on demand and supply conditions. During peak hours, high demand and limited driver availability cause prices to increase (surge pricing).

## 2.The goal of this project :

The goal of this project is to build an AI-powered dynamic pricing model that predicts the estimated ride price using factors such as:

\*number of riders requesting rides

\*number of available drivers

\*ride duration

\*vehicle type

\*location category

\*List item

## 3.Project Objectives

- Analyse ride demand and supply patterns.
- Build a machine learning model to predict ride prices.
- Understand how different factors influence ride pricing.
- Develop an interactive GUI to predict ride price.
- Demonstrate an end-to-end AI/ML pipeline

## Step 1 — Import Libraries

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.ensemble import RandomForestRegressor
7 from sklearn.preprocessing import OneHotEncoder
8 from sklearn.compose import ColumnTransformer
9 from sklearn.pipeline import Pipeline
10 from sklearn.metrics import mean_absolute_error
11
12 import joblib
```

## Step 2 — Load Dataset

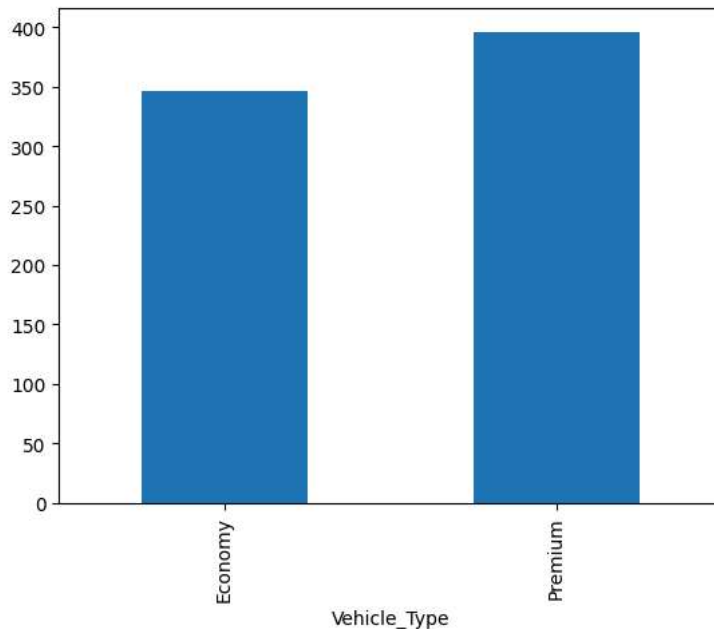
```
1 df = pd.read_csv("dynamic_pricing.csv")
2
3 df.head()
```

|   | Number_of_Riders | Number_of_Drivers | Location_Category | Customer_Loyalty_Status | Number_of_Past_Rides | Average_Ratings | Time_ |
|---|------------------|-------------------|-------------------|-------------------------|----------------------|-----------------|-------|
| 0 | 90               | 45                | Urban             | Silver                  | 13                   | 4.47            |       |
| 1 | 58               | 39                | Suburban          | Silver                  | 72                   | 4.06            |       |
| 2 | 42               | 31                | Rural             | Silver                  | 0                    | 3.99            |       |
| 3 | 89               | 28                | Rural             | Regular                 | 67                   | 4.31            |       |
| 4 | 78               | 22                | Rural             | Regular                 | 74                   | 3.77            |       |

### Step 3 — Data Exploration

```
1 df.groupby("Vehicle_Type")["Historical_Cost_of_Ride"].mean().plot(kind="bar")
```

<Axes: xlabel='Vehicle\_Type'>



## Business Insights

- Ride prices increase as rider demand grows, indicating the presence of surge pricing during high demand periods.
- Lower driver availability compared to riders leads to higher ride costs to balance supply and demand.
- Premium vehicle types consistently show higher average ride prices than economy vehicles, reflecting service tier pricing.

### How does rider demand affect ride pricing?

```

1 plt.figure(figsize=(10,6))
2
3 plt.scatter(
4     df["Number_of_Riders"],
5     df["Historical_Cost_of_Ride"],
6     s=df["Number_of_Drivers"]*5,
7     c=df["Number_of_Drivers"],
8     cmap="coolwarm",
9     alpha=0.6
10 )
11
12 plt.colorbar(label="Number of Drivers")
13
14 plt.title("Demand vs Price with Driver Availability", fontsize=16)
15 plt.xlabel("Number of Riders")
16 plt.ylabel("Ride Price")

```

```

17
18 plt.grid(alpha=0.3)
19
20 plt.show()

```



## Business Insight:

As the number of riders increases, ride prices tend to rise, indicating demand-based surge pricing used to balance supply and demand.

## ✓ How does driver availability affect ride pricing?

```

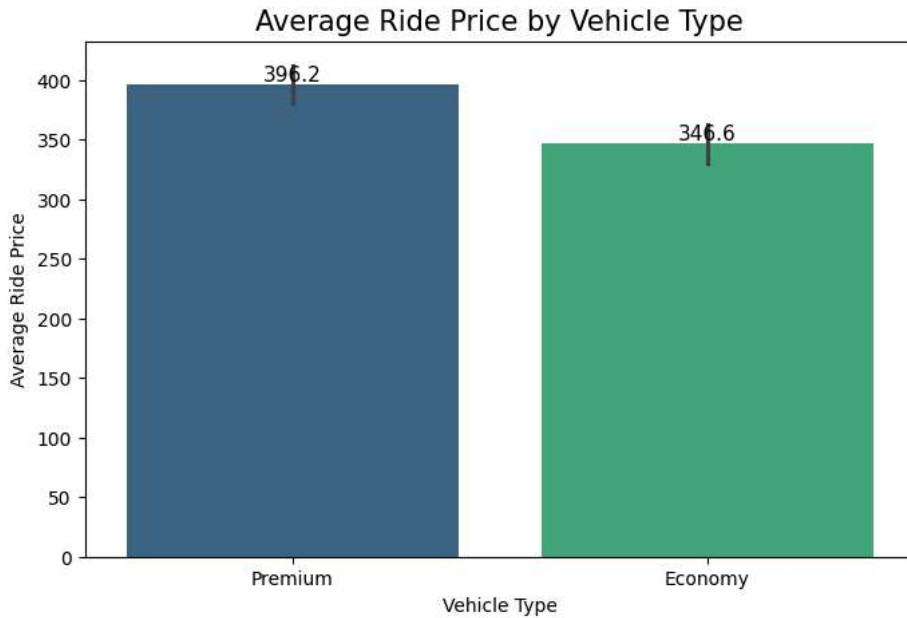
1 plt.figure(figsize=(8,5))
2
3 ax = sns.barplot(
4     data=df,
5     x="Vehicle_Type",
6     y="Historical_Cost_of_Ride",
7     palette="viridis"
8 )
9
10 for p in ax.patches:
11     ax.annotate(
12         f'{p.get_height():.1f}',
13         (p.get_x() + p.get_width()/2., p.get_height()),
14         ha='center',
15         va='bottom',
16         fontsize=11
17     )
18
19 plt.title("Average Ride Price by Vehicle Type", fontsize=15)
20 plt.xlabel("Vehicle Type")
21 plt.ylabel("Average Ride Price")
22
23 plt.show()

```

```
/tmp/ipykernel_3906/2260647976.py:3: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
ax = sns.barplot(
```



## Step 4— Feature & Target

```
1 X = df.drop("Historical_Cost_of_Ride", axis=1)
2 y = df["Historical_Cost_of_Ride"]
```

### Business Insight:

By selecting relevant features like demand, supply, and vehicle type, the model can learn the key factors that influence ride pricing and make accurate price predictions.

## How do we convert categorical data into numerical format for the model?

```
1 categorical = ["Vehicle_Type", "Location_Category"]
2 numeric = ["Number_of_Riders", "Number_of_Drivers", "Expected_Ride_Duration"]
3
4 preprocessor = ColumnTransformer(
5     transformers=[
6         ("cat", OneHotEncoder(), categorical),
7         ("num", "passthrough", numeric)
8     ]
9 )
```

## Step 5 — Model Creation

```
1 model = RandomForestRegressor(n_estimators=200)
```

### Business Insight:

Used Random Forest Regression with 200 trees to capture complex patterns between ride demand factors and ride prices, improving prediction accuracy.

## ▼ Step 6 — Build Pipeline

```
1 pipeline = Pipeline([
2     ("preprocessor", preprocessor),
3     ("model", model)
4 ])
```

Built a machine learning pipeline to automate preprocessing and model training, ensuring consistent data transformation and improving model reliability.

## ▼ Step 7 — Train/Test Split

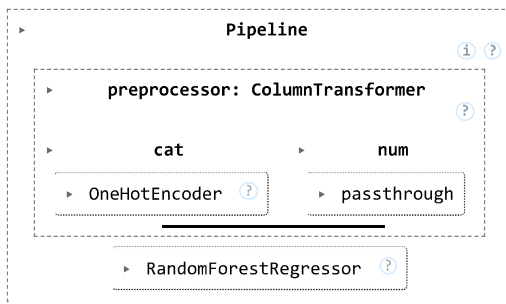
```
1 X_train,X_test,y_train,y_test = train_test_split(
2     X,y,test_size=0.2,random_state=42
3 )
```

### Project Insight:

Split the dataset into 80% training and 20% testing data to ensure the model learns patterns from historical ride data while validating its performance on unseen rides.

## ▼ Step 8 — Train Model

```
1 pipeline.fit(X_train,y_train)
```



### Business Insight :

The model was trained on historical ride data to learn patterns between demand factors (riders, drivers, distance) and ride prices.

## ▼ Step 9 — Model Evaluation

```
1 pred = pipeline.predict(X_test)
2
3 mae = mean_absolute_error(y_test,pred)
4
5 print("Model Error:",mae)
```

Model Error: 56.41583516309414

### \*\*Business Insight: \*\*

Evaluated model performance using Mean Absolute Error to measure the average difference between predicted and actual ride prices.

## Step 12 — Save the Model

```
1 joblib.dump(pipeline,"pricing_model.pkl")
```

```
['pricing_model.pkl']
```

## Step 13 — Test Prediction

```
1 sample = pd.DataFrame({
2     "Number_of_Riders":[40],
3     "Number_of_Drivers":[20],
4     "Expected_Ride_Duration":[30],
5     "Vehicle_Type":["Economy"],
6     "Location_Category":["Urban"]
7 })
8
9 pipeline.predict(sample)
```

```
array([95.1639005])
```

```
1 !pip install ipywidgets
```

```
Requirement already satisfied: ipywidgets in /usr/local/lib/python3.12/dist-packages (7.7.1)
Requirement already satisfied: ipykernel>=4.5.1 in /usr/local/lib/python3.12/dist-packages (from ipywidgets) (6.17.1)
Requirement already satisfied: ipython-genutils<=0.2.0 in /usr/local/lib/python3.12/dist-packages (from ipywidgets) (0.2.0)
Requirement already satisfied: traitlets>=4.3.1 in /usr/local/lib/python3.12/dist-packages (from ipywidgets) (5.7.1)
Requirement already satisfied: widgetsnbextension<=3.6.0 in /usr/local/lib/python3.12/dist-packages (from ipywidgets) (3.6.10)
Requirement already satisfied: ipython>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from ipywidgets) (7.34.0)
Requirement already satisfied: jupyterlab-widgets>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from ipywidgets) (3.0.16)
Requirement already satisfied: debugpy>=1.0 in /usr/local/lib/python3.12/dist-packages (from ipykernel>=4.5.1->ipywidgets) (1.8.1)
Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/python3.12/dist-packages (from ipykernel>=4.5.1->ipywidgets) (6.4.0)
Requirement already satisfied: matplotlib-inline>=0.1 in /usr/local/lib/python3.12/dist-packages (from ipykernel>=4.5.1->ipywidgets) (0.1.7)
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.12/dist-packages (from ipykernel>=4.5.1->ipywidgets) (1.6.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from ipykernel>=4.5.1->ipywidgets) (26.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from ipykernel>=4.5.1->ipywidgets) (5.9.5)
Requirement already satisfied: pyzmq>=17 in /usr/local/lib/python3.12/dist-packages (from ipykernel>=4.5.1->ipywidgets) (26.2.0)
Requirement already satisfied: tornado>=6.1 in /usr/local/lib/python3.12/dist-packages (from ipykernel>=4.5.1->ipywidgets) (6.4.0)
Requirement already satisfied: setuptools>=18.5 in /usr/local/lib/python3.12/dist-packages (from ipython>=4.0.0->ipywidgets) (75.8.0)
Collecting jedi<=0.16 (from ipython>=4.0.0->ipywidgets)
  Downloading jedi-0.19.2-py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-packages (from ipython>=4.0.0->ipywidgets) (4.4.2)
Requirement already satisfied: pickleshare in /usr/local/lib/python3.12/dist-packages (from ipython>=4.0.0->ipywidgets) (0.7.5)
Requirement already satisfied: prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from ipython>=4.0.0->ipywidgets) (3.0.48)
Requirement already satisfied: pygments in /usr/local/lib/python3.12/dist-packages (from ipython>=4.0.0->ipywidgets) (2.20.0)
Requirement already satisfied: backcall in /usr/local/lib/python3.12/dist-packages (from ipython>=4.0.0->ipywidgets) (0.2.0)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.12/dist-packages (from ipython>=4.0.0->ipywidgets) (4.9.0)
Requirement already satisfied: notebook>=4.4.1 in /usr/local/lib/python3.12/dist-packages (from widgetsnbextension<=3.6.0->ipywidgets) (4.4.1)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in /usr/local/lib/python3.12/dist-packages (from jedi<=0.16->ipython>=4.0.0->ipywidgets) (0.9.1)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-packages (from jupyter-client>=6.1.12->ipykernel>=4.5.1->ipywidgets) (3.7.0)
Requirement already satisfied: jupyter-core>=4.9.2 in /usr/local/lib/python3.12/dist-packages (from jupyter-client>=6.1.12->ipykernel>=4.5.1->ipywidgets) (5.7.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from jupyter-client>=6.1.12->ipykernel>=4.5.1->ipywidgets) (2.9.0.post0)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (3.1.6)
Requirement already satisfied: argon2-cffi in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (25.1.0)
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (5.10.4)
Requirement already satisfied: nbconvert>=5 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (7.16.0)
Requirement already satisfied: Send2Trash>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (1.9.0)
Requirement already satisfied: terminado>=0.8.3 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (0.18.0)
Requirement already satisfied: prometheus-client in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (0.21.0)
Requirement already satisfied: nbclassic>=0.4.7 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (0.5.3)
Requirement already satisfied: ptyprocess>=0.5 in /usr/local/lib/python3.12/dist-packages (from pexpect>4.3->ipython>=4.0.0->ipywidgets) (0.7.0)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0->ipython>=4.0.0->ipywidgets) (0.2.13)
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core>=4.9.2->ipykernel>=4.5.1->ipywidgets) (4.3.6)
Requirement already satisfied: notebook-shim>=0.2.3 in /usr/local/lib/python3.12/dist-packages (from nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (0.2.4)
Requirement already satisfied: BeautifulSoup4 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (4.12.3)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (6.1.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (0.7.1)
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (3.0.2)
Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (3.1.2)
Requirement already satisfied: nbclint>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (0.5.0)
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (1.5.1)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (2.21.0)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (4.23.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->jupyter-client>=6.1.12->ipykernel>=4.5.1->ipywidgets) (1.17.0)
Requirement already satisfied: argon2-cffi-bindings in /usr/local/lib/python3.12/dist-packages (from argon2-cffi->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (21.2.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension<=3.6.0->ipywidgets) (1.2.0)
```

Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->notebook)

Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema)

```
1 import pandas as pd
2 import joblib
3 import ipywidgets as widgets
4 from IPython.display import display
```

## ▼ Step 4 — Create GUI Inputs

```
1 riders = widgets.IntSlider(
2     value=20,
3     min=1,
4     max=100,
5     description='Riders'
6 )
7
8 drivers = widgets.IntSlider(
9     value=30,
10    min=1,
11    max=80,
12    description='Drivers'
13 )
14
15 duration = widgets.IntSlider(
16     value=25,
17     min=5,
18     max=60,
19     description='Duration'
20 )
21
22 vehicle = widgets.Dropdown(
23     options=['Economy', 'Premium', 'SUV'],
24     description='Vehicle'
25 )
26
27 location = widgets.Dropdown(
28     options=['Urban', 'Suburban', 'Rural'],
29     description='Location'
30 )
31
32 predict_button = widgets.Button(description="Predict Price")
33
34 output = widgets.Output()
```

## ▼ Step 5 — Prediction Function

```
1 def predict_price(b):
2
3     input_data = pd.DataFrame({
4         "Number_of_Riders": [riders.value],
5         "Number_of_Drivers": [drivers.value],
6         "Expected_Ride_Duration": [duration.value],
7
8         "Vehicle_Economy": [1 if vehicle.value=="Economy" else 0],
9         "Vehicle_Premium": [1 if vehicle.value=="Premium" else 0],
10        "Vehicle_SUV": [1 if vehicle.value=="SUV" else 0],
11
12        "Location_Urban": [1 if location.value=="Urban" else 0],
13        "Location_Suburban": [1 if location.value=="Suburban" else 0],
14        "Location_Rural": [1 if location.value=="Rural" else 0]
15    })
16
17    price = model.predict(input_data)[0]
18
19    with output:
20        output.clear_output()
21        print("Estimated Ride Price: ₹", round(price,2))
```

▼

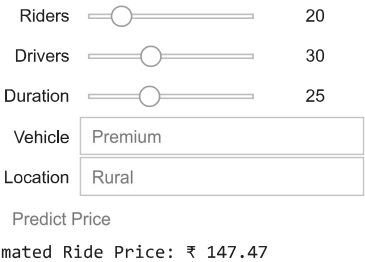
Connect Button

```
1 predict_button.on_click(predict_price)
```

▼

Step 7 — Display GUI

```
1 display(riders, drivers, duration, vehicle, location, predict_button, output)
```



Key Insights:

- Some insights discovered during analysis:
- Ride prices increase when rider demand increases
  - Prices decrease when more drivers are available
  - Premium and SUV rides cost significantly more than economy rides
  - Urban locations show higher ride prices compared to rural areas

▼

Future Improvements:

- Possible enhancements:
- Real-time surge pricing algorithm
  - demand forecasting model
  - deep learning price prediction
  - live dashboard using Streamlit
  - API deployment

Double-click (or enter) to edit